第 1-3 讲 • 代码是怎么变成软件的

> **模块 1 • 程序到底是什么** ｜ 建议用时 **40 分钟**（阅读 24 + 动手 16）
> 前置要求：完成 1-1、1-2

本节目标

读完并做完这一讲，你能：

1. 讲清**源代码 → 构建 → 打包**这条三级流水线，以及每一级"输入什么、输出什么"。
2. 解释为什么**电脑看不懂你写的代码**，需要先"翻译"。
3. 理解**TypeScript 和 JavaScript 的关系**，以及为什么项目用前者。
4. 说出 `src`、`out`、`release` 三个目录的区别，知道改代码后要跑哪一步才能看到效果。
5. **亲眼验证**："74 个源码文件"变成了"3 个产物文件"——以及这个压缩过程意味着什么。

一、三级流水线

1.1 现实比喻：从手稿到出版书

阶段	比喻	你的项目
源代码	作者的手稿	`src\` 里的 `.ts` / `.tsx` 文件
构建	排版、校对、印刷成册	`npm run build` → 输出到 `out\`
打包	装订、加封面、装箱出厂	`npm run build:win` → 输出到 `release\`

为什么要有这么多步，不能直接用手稿：

- **手稿只有作者看得懂**（你自己写的 TypeScript，浏览器和 Node 都不认）；
- **读者要的是成品书**（用户要的是能双击运行的 `.exe`）。

每一级的产物：

阶段	命令	输入	输出	谁能用
开发	`npm run dev`	`src\`	内存里临时构建	你自己调试
构建	`npm run build`	`src\`	`out\`	开发运行、下一步打包
打包	`npm run build:win`	`out\` + 资源	`release\` 里的安装包	**任何人**

记住这张表，它解释了后面所有"我改了代码但没效果"的疑问。

1.2 为什么电脑看不懂你写的代码

计算机的 CPU 只认**机器指令**（一串二进制数字）。人类不可能直接写那个，所以有了层层"翻译"：

```
你的代码（TypeScript）
      ↓ 编译（翻译）
JavaScript
      ↓ 由 Node / Chromium 执行
机器指令
      ↓
CPU 真的在干活
```

为什么要翻译两次：因为 TypeScript 是"给人看的 JavaScript"——它多了类型标注，能提前发现错误，但浏览器不认它，必须先转成普通 JavaScript。

二、TypeScript 是什么

2.1 一句话解释

TypeScript = JavaScript + 类型标注。

看两个版本的同一个函数：

```
// JavaScript：能跑，但没人拦你犯错
function addMoney(a, b) {
  return a + b
}
addMoney("12", 5)   // 结果 "125" —— 字符串拼接！不是加法！但没人报错
```

```
// TypeScript：写错立刻被发现
function addMoney(a: number, b: number): number {
  return a + b
}
addMoney("12", 5)   // ❌ 红字报错：string 不能传给 number
```

这就是"类型"的价值：把"你写错了"这件事从**用户使用时**提前到**你写代码时**。

对记账软件尤其重要：金额是分（整数），如果某处不小心把"12.5 元"当成了"12.5 分"，用 JavaScript 会静默算错，用 TypeScript 会在写代码时就拦住你。**钱算错是记账软件最不可接受的 bug**，所以这个项目选了 TypeScript。

2.2 项目里的真实情况

事实	数值
源代码文件（`.ts` + `.tsx`）	**100 个**（74 个 `.ts` + 26 个 `.tsx`）
构建后的产物文件（`.js`）	**3 个**

为什么 100 个变成 3 个？因为构建工具会把它们**合并**（bundling）——把所有 `import` 进来的文件拼成一个大文件，这样运行时不用一个个去加载，更快。

> **这是个重要的心理准备**：你在 `src\` 里改了代码， `out\` 里的产物**不会自动更新**。必须重新构建（或处于 `dev` 模式）。**记错这一点，你会对着一个"没变化"的窗口怀疑人生。**

三、在项目里看真实文件

3.1 三个关键目录

```
记账簿\
├─ src\          ← 你写代码的地方（源代码）
│   ├─ main\       主进程（管数据、系统）
│   ├─ preload\    桥梁（第 2-2 讲）
│   ├─ renderer\   界面（用户看到的）
│   └─ shared\     两边共用
├─ out\          ← 构建产物（自动生成，不要手改）
│   ├─ main\index.js
│   ├─ preload\index.js
│   └─ renderer\index.html + assets\
├─ release\      ← 最终安装包（能发给别人）
└─ node_modules\ ← 第三方零件
```

一条清晰的规矩：

目录	你该动它吗	理由
`src\`	✅ **只在这里写代码**	这是唯一的"真相来源"
`out\`	❌ 不动	每次构建都会被覆盖，改了白改
`release\`	❌ 不动	每次打包都会重新生成
`node_modules\`	❌ 不动	第三方零件

这是新手最容易犯的错：以为 `out\` 或 `release\` 里的东西是"需要维护的"，跑去改它们。**记住：只有 `src\` 是你的，其它都是"生成出来的"。**

3.2 构建配置：谁告诉工具"从哪开始"

打开 `electron.vite.config.ts`，你会看到三段（简化）：

```
main: {
  rollupOptions: { input: { index: resolve(__dirname, 'src/main/index.ts') } }
},
preload: {
  rollupOptions: { input: { index: resolve(__dirname, 'src/preload/index.ts') } }
},
renderer: {
  root: resolve(__dirname, 'src/renderer'),
  rollupOptions: { input: { index: resolve(__dirname, 'src/renderer/index.html') } }
}
```

这段配置在说：

> "构建的时候，从这三个文件出发：
> ① `src/main/index.ts`（主进程入口）
> ② `src/preload/index.ts`（桥梁）
> ③ `src/renderer/index.html`（界面入口）
> **顺着它们里面的 import 一路找下去，把所有用到的文件都打包进来。**"

这就是为什么 100 个文件能变成 3 个：工具从三个入口出发，把"被引用到的"文件全收进来，合并。

顺带一个领悟：没被任何入口引用到的文件，**不会进产物**。所以你如果新建了一个文件但没 `import` 它，它就不会生效——这是"我写了代码但没反应"的另一个常见原因。

3.3 打包配置：安装包长什么样

打开 `electron-builder.yml`，关键字段：

```
appId: com.dailyledger.app          # 应用的唯一标识（类似身份证号）
productName: 记账簿                  # 显示给用户的名字
directories: ...                    # 输出到哪
files: ...                          # 哪些文件要装进安装包
```

`appId` **为什么重要**：它是系统识别"这是哪个软件"的依据。**一旦发布就不能改**——改了系统会认为这是一个全新的软件，用户的设置和数据都对不上。（这也是为什么我在上架清单里让你先定好它。）

四、动手任务

步骤 1：亲眼看到"源码"和"产物"

```
cd "C:\Users\34123\Desktop\记账簿"

# 数一数源码文件
(Get-ChildItem src -Recurse -File -Include *.ts,*.tsx | Measure-Object).Count

# 数一数构建产物
(Get-ChildItem out -Recurse -File -Filter *.js -ErrorAction SilentlyContinue | Measure-Obj
```

预期：第一个数字约 **100**，第二个是 **3**。

这就是"打包合并"的直观证据。

步骤 2：对比源码和产物（关键体验）

先看**源码**（人能看懂）：

```
Get-Content src\preload\index.ts -TotalCount 20
```

你会看到带类型标注的代码，有 `import`、有 `: string` 这样的记号。

再看**产物**（机器要的）：

```
Get-Content out\preload\index.js -TotalCount 20
```

观察差异（下面是实测出来的真实数字）：

对比项	源码 `.ts`	产物 `.js`
类型标注 `: string` 出现次数	**31 次**	**0 次**（全被删掉）
注释	有	大部分被删掉
缩进与换行	讲究、好读	紧凑、省空间
人能看懂吗	能	勉强能，但很丑

怎么自己验证这个数字：

```
(Select-String -Path src\preload\index.ts -Pattern ": string" -AllMatches).Count
(Select-String -Path out\preload\index.js -Pattern ": string" -AllMatches).Count
```

第一条会输出 31，第二条输出 0。 这就是"类型标注只是给开发时用的，运行时根本不存在"的铁证。

结论：构建做了三件事——**翻译**（TS→JS）、**删除**（类型和注释，运行时用不到）、**压缩**（少占空间）。所以产物**不是给你改的。**

步骤 3：验证"改 src 会让 out 变旧"

```
# 记录 out 的时间
(Get-Item out\main\index.js).LastWriteTime

# 记录 src 的时间
(Get-Item src\main\index.ts).LastWriteTime
```

现在去 `src\main\index.ts` 里随便加一行注释（用 VS Code 或记事本），存盘。然后再看：

```
(Get-Item out\main\index.js).LastWriteTime
```

预期：`out` 的时间**没变** —— 它不会自动跟着更新。

然后执行构建：

```
npm run build
```

构建完再看一次 `out\main\index.js` 的时间，**这次变了。**

这一课的意义：你亲手验证了"改源码 ≠ 改产物"。以后遇到"我改了怎么没效果"，第一件事就是问自己：**我重新构建了吗?**

> 补充：`npm run dev` 模式下会自动重建，所以开发时你感觉不到这一步。但一旦离开 dev 模式（比如你自己打包），就必须手动构建。**理解了这个机制，你就不会在打包时被"改了没生效"搞糊涂。**

步骤 4：看看打包配置

```
Get-Content electron-builder.yml | Select-String "^(appId|productName)"
```

预期看到：`appId: com.dailyledger.app` 和 `productName: 记账簿`。

思考题：如果你把 `productName` 从「记账簿」改成「我的账本」，重新打包后安装包叫什么名字?

答案：文件名会变成 `我的账本-0.1.0-portable.exe`，安装后的程序名也变了。**但 `appId` 没变**，所以系统仍认为它是同一个软件（用户数据不会丢）。

这就说明了两者的分工：`productName` 是"给人看的名字"（可以改），`appId` 是"给系统看的身份证"（不能改）。

卡住了怎么办

现象	原因	怎么办
`npm run build` 报错	源码有语法错误	先跑 `npm run typecheck` 看具体哪个文件哪一行
构建成功但软件行为没变	你看的是旧的 `release` 里的包	重新打包（`npm run build:win`）才会更新安装包
`out` 目录不存在	从没构建过	跑一次 `npm run build`
数出来产物不止 3 个 js	统计口径不同（如含 assets 里的分片）	正常，重点是"很多源码 → 很少产物"这个事实

五、自测题

1. `src`、`out`、`release` 三个目录分别是什么？你只该改哪一个？

2. TypeScript 和 JavaScript 是什么关系？项目为什么用 TypeScript？

3. 100 个源码文件为什么能变成 3 个产物文件？

4. 你改了 `src` 里的代码，但运行软件发现没变化。最可能的原因是什么？

5. `productName` 和 `appId` 有什么区别？为什么后者不能随便改？

答案与解析

1. `src` 是**源代码**（你写的）；`out` 是**构建产物**（自动生成）；`release` 是**最终安装包**（能发给别人）。 → **只该改** `src`。另外两个每次构建/打包都会被覆盖，改了也是白改。

2. TypeScript = **JavaScript + 类型标注**，是"给人看的 JavaScript"。浏览器和 Node 不认 TS，所以要先编译成 JS。 → 项目用它是因为**类型能在写代码时就发现错误**。对记账软件尤其关键：金额算错是最不可接受的 bug，类型能提前拦住。

3. 因为构建工具会**从入口文件出发，顺着 `import` 把所有用到的文件合并成一个大文件**（bundling）。 → 反过来说：**没被任何入口引用到的文件不会进产物**——所以你新建文件后必须 `import` 它才会生效。

4. 忘了重新构建（或者没在 `dev` 模式下）。 → `out` 不会自动跟着 `src` 更新。`npm run dev` 会自动重建，但手动打包时必须先 `npm run build`。

5. `productName` 是**显示给用户的名字**（安装包文件名、程序名），**可以改**；`appId` 是**系统识别软件的身份证，发布后不能改**——改了系统会当成一个全新软件，用户原有的设置和数据就对不上了。

六、术语表

术语	一句话解释
源代码 / Source	你写的代码，在 `src\`，人和机器都不直接执行它
构建 / Build	把源代码翻译、合并、压缩成可运行产物的过程
打包 / Package	把构建产物 + 运行环境 + 资源装成安装包
编译 / Compile	把一种语言翻译成另一种（这里是 TS → JS）
TypeScript（TS）	JavaScript + 类型标注，写错会被提前拦下
JavaScript（JS）	浏览器和 Node 能直接执行的语言
类型 / Type	声明"这个值是数字/文字/对象"，用来提前发现错误
打包合并 / Bundling	把许多源文件合并成少数产物文件
入口 / Entry	构建从哪个文件开始找（本项目有三个入口）
产物 / Artifact	构建或打包生成的文件，**不要手改**
`out\`	构建产物目录（自动生成）
`release\`	安装包输出目录（自动生成）
`appId`	应用的唯一标识，**发布后不可改**
`productName`	显示给用户的应用名，可以改

七、本讲与项目的对应

- 你查看/改动的文件：`src\main\index.ts`（加了一行注释）、`electron.vite.config.ts`、`electron-builder.yml`
- 你执行的命令：`npm run build`（会重建 `out\`，**不影响你的记账数据**）
- 上一讲：1-2 一切都是文件 ｜ **下一讲进入模块 2：2-1 主角登场——主进程与渲染进程**

讲义版本：v1 ｜ 对应项目版本：0.1.0